

For example: if there is no list's instance variable before, write statements in main program to create the instance variable mylist with ordered inputs are 1, 2, 3, 4, 5 and has result outputs are 1, 2, 3, 4, 5 by using class LinkedList's methods that we learned.

First of all, I suggest you to use pencil or pen to underline or circle the **important parts of Question**, ask yourself,

1. What is the question want to do?
2. What are inputs and what are outputs?

The wrong solution is:

```
mylist = LinkedList()
mylist.add(1)
mylist.add(2)
mylist.add(3)
mylist.add(4)
mylist.add(5)
```

Why is it wrong?

[คำเตือน 0 คะแนน] ทำไมคำตอบนี้ถึงผิด (Why is it wrong?):

เมธอด add(item) ของคลาส LinkedList คือการแทรกที่หัวลิสต์ (Insert at Beginning / Prepend) เมื่อเรียก add(1) ถึง add(5) โหนดใหม่จะถูกดันมาอยู่ข้างหน้าสุดเสมอ ทำให้ลิสต์กลายเป็น: 5 -> 4 -> 3 -> 2 -> 1 ซึ่งขัดแย้งกับโจทย์ที่สั่งว่า 'has result outputs are 1, 2, 3, 4, 5' (Ascending Order) จึงได้ 0 คะแนน

The right solution is:

[เฉลยเป็นทางการ] 2 แนวทางคำตอบที่ถูกต้องตามมาตรฐานอาจารย์ (The Right Solutions):

* แนวทางที่ 1: สลับลำดับการเรียก add() ย้อนหลังจาก 5 ลงมาถึง 1 (Reverse Input Order)

```
mylist = LinkedList()
mylist.add(5); mylist.add(4); mylist.add(3); mylist.add(2); mylist.add(1)
-> ผลลัพธ์: 1 -> 2 -> 3 -> 4 -> 5 (ได้ผลลัพธ์ถูกต้องตรงตามโจทย์ต้องการทุกประการ)
```

* แนวทางที่ 2: ใช้ add(1) เป็นโหนดแรก แล้วใช้ insert(pos, item) หรือ append(item)

```
mylist = LinkedList()
mylist.add(1)
mylist.insert(1, 2); mylist.insert(2, 3); mylist.insert(3, 4); mylist.insert(4, 5)
-> ผลลัพธ์: 1 -> 2 -> 3 -> 4 -> 5
```

For example: if there is the exist list's instance variable mylist which has one input 3 already,
write statements in main program to create the instance variable mylist with ordered inputs are 1, 2, 4, 5 and has result outputs are 1, 2, 3, 4, 5 by using class LinkedList's methods that we learned.
The solution is:

mylist = LinkedList() **X** no need to create a new instance variable
mylist.add(1)
mylist.add(2) **X** not use add method

[วิเคราะห์กับดักใจหาย] ทำไมห้ามสร้าง LinkedList() ใหม่ และห้ามใช้ add(2)?:

- * สถานะเริ่มต้นในหน่วยความจำ: มีโหนด [3] อยู่แล้ว (head -> [3] -> None)
- * กับดักที่ 1: mylist = LinkedList() [ได้ 0 คะแนน!] เพราะเป็นการ new ออบเจกต์ใหม่ทับของเดิม ทำให้พอยน์เตอร์เดิมหลุด โหนด [3] หายไปจากหน่วยความจำทันที (Lost Reference / Memory Leak)
- * กับดักที่ 2: mylist.add(2) ต่อจาก add(1) [ได้ 0 คะแนน!] เพราะ add() แทรกที่หัวเสมอ หากเรียก add(1) แล้วตามด้วย add(2) ลิสต์จะกลายเป็น [2] -> [1] -> [3] ซึ่งผิดลำดับที่โจทย์สั่ง

[เฉลยเป็นทางการ] คำสั่งที่ถูกต้องเพื่อสร้าง [1, 2, 3, 4, 5] จากโหนด [3] เดิม:

- * ขั้นตอนที่ 1 (แทรก 1 ที่หัวลิสต์): mylist.add(1) -> ลิสต์กลายเป็น: [1] -> [3]
 - * ขั้นตอนที่ 2 (แทรก 2 ระหว่าง 1 กับ 3): mylist.insert(1, 2) -> ลิสต์กลายเป็น: [1] -> [2] -> [3]
 - * ขั้นตอนที่ 3 (แทรก 4 ต่อท้าย 3): mylist.insert(3, 4) -> ลิสต์กลายเป็น: [1] -> [2] -> [3] -> [4]
 - * ขั้นตอนที่ 4 (แทรก 5 ต่อท้าย 4): mylist.insert(4, 5) -> ลิสต์กลายเป็น: [1] -> [2] -> [3] -> [4] -> [5]
- >> สรุปชุดคำสั่งภาษา Python ที่ต้องเขียนตอบ (The Right Statements):
- ```
mylist.add(1) # Insert 1 at index 0 (Head)
mylist.insert(1, 2) # Insert 2 at index 1
mylist.insert(3, 4) # Insert 4 at index 3
mylist.insert(4, 5) # Insert 5 at index 4
```
- > ผลลัพธ์สุดท้าย: 1 -> 2 -> 3 -> 4 -> 5 ตรงตามเงื่อนไขข้อสอบทุกประการ ได้คะแนนเต็ม 100%

[โจทย์เสริมและเทคนิคห้องเรียน] การสลับพอยน์เตอร์ 4 โหนดโดยไม่ทำโหนดหลุดหาย:

- \* โจทย์เสริมในห้องเรียน: สลับโหนด 1 -> 4 -> 3 -> 2 -> 5 ให้เป็น 1 -> 2 -> 3 -> 4 -> 5
  - \* ลำดับการปรับพอยน์เตอร์เพื่อป้องกันโหนดหลุดหาย (No Dangling Pointer):
    1. node2.next = node5 # ขั้นที่ 1: เชื่อม node2 ไปที่ node5 ก่อน
    2. node3.next = node2 # ขั้นที่ 2: ให้ node3 ชีกลับไปที่ node2
    3. node4.next = node3 # ขั้นที่ 3: ให้ node4 ชีมาที่ node3
    4. node1.next = node4 # ขั้นที่ 4: ปรับหัวลิสต์ node1 ให้ชี้ไปที่ node4 เป็นตัวแรก
- > กฎเหล็ก: ต้องต่อปลายสายก่อนตัดเส้นเชื่อมเดิมเสมอ เพื่อไม่ให้โหนดสูญหายกลางทาง